

UD04.1: Datos avanzados y colecciones

Contenido

1. Estructuras de datos en Python	2
1.1 Estructuras estáticas y dinámicas	2
1.2 Introducción a <i>arrays</i> con la librería <i>array</i>	2
1.3 Matrices multidimensionales: listas de listas y <i>numpy</i>	3
2. Cadenas de caracteres (str).....	5
2.1 Operaciones básicas con strings	5
2.2 Métodos de cadena útiles	5
2.3 Expresiones regulares (RegEx)	6
2.4 Búsqueda y validación de patrones	6
3. Colecciones en Python	7
3.1 Listas: características y uso	7
3.2 Tuplas (tuple): características y uso	8
3.3 Conjuntos (set): uso y operaciones	9
3.4 Diccionarios (dict): claves y valores	9
3.5 Comparación entre colecciones.....	10
3.6 Iteradores y recorrido de colecciones	10
4. Genericidad en Python.....	11
4.1 Introducción a la programación genérica	11
4.2 Clases y funciones genéricas con <i>typing</i>	11
4.3 Ventajas y ejemplos prácticos	12
5. Operaciones agregadas sobre colecciones	13
5.1 Filtrado de elementos	13
5.2 Reducción de datos.....	13
5.3 Recolección y transformación (map)	14
5.4 Comprensiones de colecciones: list, set y dict	14

1. Estructuras de datos en Python

Una estructura de datos es una forma organizada de almacenar datos para que puedan ser utilizados de manera eficiente.

1.1 Estructuras estáticas y dinámicas

- **Estructuras estáticas:** Su tamaño se define al crearlas y no se puede modificar. En Python, las tuplas y las cadenas de caracteres se comportan de manera estática en cuanto a su inmutabilidad.
- **Estructuras dinámicas:** Su tamaño puede variar durante la ejecución del programa. Las listas y los diccionarios son ejemplos de estructuras dinámicas. Python gestiona internamente la asignación de memoria para acomodar nuevos elementos.

1.2 Introducción a *arrays* con la librería *array*

El módulo *array* de la biblioteca estándar de Python permite crear *arrays* de tipo estático de forma más eficiente en memoria que las listas, ya que todos sus elementos deben ser del mismo tipo.

Pero es importante destacar que esta librería es poco usada en la práctica, ya que la mayoría de los desarrolladores optan por las listas (*list*) para datos de tipo mixto o por la librería *numpy* para datos numéricos de gran volumen debido a su versatilidad y rendimiento.

Ejemplo:

```
import array
```

```
# Crea un array de enteros 'i' de tamaño fijo
```

```
mi_array = array.array('i', [1, 2, 3, 4, 5])
```

```
print(mi_array)
```

```
# Salida: array('i', [1, 2, 3, 4, 5])
```

```
# No se puede añadir un tipo de dato diferente
```

```
# mi_array.append("hola") # Esto generaría un error
```

1.3 Matrices multidimensionales: listas de listas y numpy

En Python, una matriz se puede representar como una **lista de listas**, una estructura de datos bidimensional donde cada sublista representa una fila. Esta es una forma sencilla de empezar, pero para operaciones matemáticas y de alto rendimiento, se utiliza la librería numpy.

Creación de una matriz con listas de listas:

Ejemplo:

```
# Creación de una matriz 3x3
```

```
matriz = [
```

```
    [1, 2, 3],
```

```
    [4, 5, 6],
```

```
    [7, 8, 9]
```

```
]
```

```
print(matriz[1][2]) # Acceder al elemento de la fila 1, columna 2 (el 6)
```

```
# Salida: 6
```

Creación de matrices con numpy (la forma estándar): La librería numpy es el estándar en Python para computación numérica y es indispensable para trabajar con matrices y *arrays* multidimensionales de manera eficiente. Proporciona un objeto ndarray que es mucho más rápido y potente que las listas de listas.

Ejemplo:

```
import numpy as np
```

```
# Crear una matriz 2x3 con numpy
```

```
matriz_np = np.array([[1, 2, 3], [4, 5, 6]])
```

```
print(matriz_np)
```

```
# Salida:
```

```
# [[1 2 3]
```

```
#  [4 5 6]]
```

```
# Operaciones vectorizadas (sin necesidad de bucles)
```

```
matriz_np_2 = matriz_np * 2
```

```
print(matriz_np_2)
```

```
# Salida:
```

```
# [[ 2  4  6]
```

```
# [ 8 10 12]]
```

2. Cadenas de caracteres (str)

Las cadenas de caracteres son secuencias inmutables de caracteres.

2.1 Operaciones básicas con strings

- **Concatenación:** Se usa el operador +.

```
saludo = "Hola" + " " + "mundo" # "Hola mundo"
```

- **Repetición:** Se usa el operador *.

```
risa = "ja" * 3 # "jajaja"
```

- **Indexación y Slicing:** Acceso a caracteres y subcadenas.

```
cadena = "Python"  
print(cadena[0]) # 'P'  
print(cadena[1:4]) # 'yth'
```

2.2 Métodos de cadena útiles

Las cadenas de texto tienen numerosos métodos que facilitan su manipulación:

- `.upper()`, `.lower()`: Convertir a mayúsculas o minúsculas.
- `.strip()`: Eliminar espacios en blanco al inicio y al final.
- `.split(separador)`: Dividir la cadena en una lista de subcadenas.
- `.join(lista)`: Unir una lista de cadenas con el *string* como separador.
- `.replace(viejo, nuevo)`: Reemplazar todas las ocurrencias de una subcadena.
- `.find(subcadena)`: Buscar una subcadena y devolver su índice (o -1 si no se encuentra).

2.3 Expresiones regulares (RegEx)

Las **expresiones regulares (RegEx)** son secuencias de caracteres que forman un patrón de búsqueda. Son una herramienta muy poderosa para buscar, validar y manipular texto. La librería estándar de Python para trabajar con ellas es el módulo **re**.

Ejemplo:

```
import re
```

```
patron = r"p[aeiou]t[aeiou]" # Por ejemplo, para buscar 'pat', 'pet', 'pit', 'pot', 'put'
```

```
cadena = "El gato y el pato están en el patio."
```

```
coincidencia = re.search(patron, cadena)
```

```
if coincidencia:
```

```
    print("Coincidencia encontrada:", coincidencia.group())
```

```
# Salida: Coincidencia encontrada: pato
```

2.4 Búsqueda y validación de patrones

Con el módulo **re** se pueden realizar diferentes operaciones:

- `re.search(patron, cadena)`: Busca la primera coincidencia del patrón.
- `re.match(patron, cadena)`: Busca el patrón solo al principio de la cadena.
- `re.findall(patron, cadena)`: Devuelve una lista con todas las coincidencias.
- `re.sub(patron, reemplazo, cadena)`: Sustituye las coincidencias por una cadena de reemplazo.

3. Colecciones en Python

Las colecciones son tipos de datos que agrupan varios elementos en una sola variable.

3.1 Listas: características y uso

- **Características:** Ordenadas, mutables, pueden contener elementos de diferentes tipos y permiten duplicados. Se definen con corchetes [].
- **Uso:** Son la estructura más versátil para almacenar una secuencia de datos.

Ejemplo:

```
mi_lista = [1, "hola", True, 1]
```

Principales métodos de las listas en Python

Agregar o combinar elementos

Método	Descripción	Ejemplo
append(x)	Añade un elemento al final de la lista.	mi_lista.append(4)
extend(iterable)	Añade varios elementos desde otra lista o iterable.	mi_lista.extend([5, 6])
insert(i, x)	Inserta un elemento x en la posición i.	mi_lista.insert(1, "nuevo")

Eliminar elementos

Método	Descripción	Ejemplo
remove(x)	Elimina la primera ocurrencia de x.	mi_lista.remove("hola")
pop([i])	Elimina y devuelve el elemento en la posición i (por defecto, el último).	mi_lista.pop()
clear()	Elimina todos los elementos de la lista.	mi_lista.clear()

Buscar o contar elementos

Método	Descripción	Ejemplo
index(x)	Devuelve el índice de la primera ocurrencia de x.	mi_lista.index(1)
count(x)	Cuenta cuántas veces aparece x.	mi_lista.count(1)

Ordenar o invertir

Método	Descripción	Ejemplo
sort(reverse=False)	Ordena la lista en su lugar .	mi_lista.sort()
reverse()	Invierte el orden de los elementos.	mi_lista.reverse()

Copiar listas

Método	Descripción	Ejemplo
copy()	Crea una copia superficial de la lista.	nueva_lista = mi_lista.copy()

3.2 Tuplas (tuple): características y uso

- **Características:** Ordenadas, inmutables, pueden contener elementos de diferentes tipos y permiten duplicados. Se definen con paréntesis () o separando los valores por comas.
- **Uso:** Se utilizan para almacenar datos que no deben modificarse, como coordenadas o configuraciones fijas. Son **más rápidas y seguras** que las listas cuando no se requiere modificación.

Ejemplo:

```
mi_tupla = (10, 20, 30, 20)

print(mi_tupla[0])    # Acceder al primer elemento

print(mi_tupla.count(20)) # Contar cuántas veces aparece un valor

print(mi_tupla.index(30)) # Obtener el índice de un valor
```

Las tuplas son inmutables: esto genera error

mi_tupla[1] = 50

Creación especial de tuplas:

tupla_un_elemento = (5,) # Es necesario incluir la coma

sin_parenthesis = 1, 2, 3 # También válida

Conversión entre lista y tupla:

lista = [1, 2, 3]

tupla = tuple(lista)

print(tupla) # (1, 2, 3)

3.3 Conjuntos (set): uso y operaciones

- **Características:** No ordenados, mutables, no permiten elementos duplicados. Son ideales para eliminar duplicados o para operaciones matemáticas de conjuntos. Se definen con llaves {} o con set().
- **Operaciones:**
 - **Unión:** set1 | set2
 - **Intersección:** set1 & set2
 - **Diferencia:** set1 - set2

a = {1, 2, 3}

b = {3, 4, 5}

print(a | b) # Unión: {1, 2, 3, 4, 5}

print(a & b) # Intersección: {3}

3.4 Diccionarios (dict): claves y valores

- **Características:** Pares clave: valor. Ordenados desde Python 3.7, mutables. Las claves deben ser únicas e inmutables (cadenas, tuplas, números). Se definen con llaves {}.
- **Uso:** Ideales para almacenar datos con una estructura de clave-valor.

```

persona = {"nombre": "Juan", "edad": 30, "ciudad": "Madrid"}

print(persona["nombre"]) # Acceder a un valor

persona["edad"] = 31     # Modificar un valor

persona["email"] = "j@g.com" # Añadir un nuevo par clave-valor

```

3.5 Comparación entre colecciones

Tipo	Ordenado	Mutable	Duplicados	Uso principal
Lista (list)	Sí	Sí	Sí	Almacenar secuencias de elementos
Tupla (tuple)	Sí	No	Sí	Agrupar datos inmutables, claves de diccionarios
Conjunto (set)	No	Sí	No	Colecciones únicas, operaciones de conjuntos
Diccionario (dict)	Sí (≥ 3.7)	Sí	No (claves)	Almacenar pares clave-valor

3.6 Iteradores y recorrido de colecciones

Un **iterador** es un objeto que permite recorrer una colección. Las colecciones son objetos *iterables*, lo que significa que se pueden recorrer con un bucle `for`.

Ejemplo:

```

# Recorrer una lista

for numero in [1, 2, 3]:

    print(numero)


# Recorrer un diccionario (accediendo a claves, valores o ambos)

for clave, valor in persona.items():

    print(f"Clave: {clave}, Valor: {valor}")

```

4. Genericidad en Python

La programación genérica permite escribir código que funciona con diferentes tipos de datos sin necesidad de reescribirlo para cada tipo.

4.1 Introducción a la programación genérica

Aunque Python es un lenguaje de tipado dinámico, el uso de anotaciones de tipo se ha popularizado para mejorar la legibilidad, la robustez del código y la detección de errores. No es obligatorio, pero es una buena práctica.

4.2 Clases y funciones genéricas con typing

El módulo typing se utiliza para indicar de forma opcional los tipos de datos en el código. Esto no afecta a la ejecución, pero sí ayuda a herramientas de análisis estático y a los IDEs.

También se pueden usar anotaciones de tipo básicas sin el módulo, como : int o -> str.

```
from typing import List, Dict, Any
```

```
# Función con anotaciones de tipo básicas
```

```
def suma_numeros(a: int, b: int) -> int:
```

```
    """Suma dos enteros y devuelve un entero."""
```

```
    return a + b
```

```
# Función con anotaciones de tipo genéricas
```

```
def obtener_primer_elemento(coleccion: List[Any]) -> Any:
```

```
    """Devuelve el primer elemento de cualquier lista."""
```

```
    return coleccion[0]
```

```
# Ejemplo de uso
```

```
numeros = [10, 20, 30]
```

```
cadenas = ["a", "b", "c"]
```

```
print(obtener_primer_elemento(numeros)) # 10  
print(obtener_primer_elemento(cadenas)) # "a"
```

4.3 Ventajas y ejemplos prácticos

- **Mejora de la legibilidad:** El código es más fácil de entender.
- **Detección de errores:** Las herramientas pueden detectar errores de tipo antes de ejecutar el código.
- **Autocompletado:** Los IDEs pueden ofrecer mejores sugerencias.
- **Documentación:** Las anotaciones de tipo sirven como una forma de documentación en sí mismas.

5. Operaciones agregadas sobre colecciones

Son operaciones que procesan una colección de datos para producir un resultado.

Estas operaciones agregadas recorren las colecciones de forma interna aplicando las funciones indicadas.

5.1 Filtrado de elementos

Seleccionar elementos de una colección que cumplen una condición.

- **Con bucle for:**

```
pares = []  
for num in [1, 2, 3, 4]:  
    if num % 2 == 0:  
        pares.append(num)
```

- **Con filter():**

```
def es_par(x):  
    return x % 2 == 0
```

```
pares = list(filter(es_par, [1, 2, 3, 4]))  
print(pares) # [2, 4]
```

5.2 Reducción de datos

Combinar todos los elementos de una colección en un único valor.

- **Con sum():**

```
total = sum([1, 2, 3, 4]) # 10
```

- **Con `functools.reduce()`:**

```
from functools import reduce

total = reduce(lambda x, y: x + y, [1, 2, 3, 4]) # 10
```

5.3 Recolección y transformación (`map`)

Aplicar una función a cada elemento de una colección para crear una nueva colección.

- **Con bucle `for`:**

```
cuadrados = []

for num in [1, 2, 3]:
    cuadrados.append(num * num)
```

- **Con `map()`:**

```
cuadrados = list(map(lambda x: x * x, [1, 2, 3]))

# Salida: [1, 4, 9]
```

5.4 Comprensiones de colecciones: `list`, `set` y `dict`

Las **comprensiones** son una forma concisa de realizar operaciones de filtrado y transformación en una sola línea. En la mayoría de los casos, son más legibles y eficientes que las funciones `map()` y `filter()`.

- **Comprensión de listas:**

```
pares = [num for num in [1, 2, 3, 4] if num % 2 == 0]

# Salida: [2, 4]

cuadrados = [num * num for num in [1, 2, 3]]

# Salida: [1, 4, 9]
```

- **Comprensión de conjuntos:**

```
pares_unicos = {num for num in [1, 2, 2, 3, 4] if num % 2 == 0}
```

```
# Salida: {2, 4}
```

- **Comprensión de diccionarios:**

```
cuadrados_dict = {num: num*num for num in [1, 2, 3]}
```

```
# Salida: {1: 1, 2: 4, 3: 9}
```